



ICS3U Resource Book

Revised August 2013

Chapter 7

Object Interaction

Until now, we have been studying classes and objects more or less in isolation but an important aspect of objects is the many ways that they can interact with each other. In this chapter, we examine some of the relationships that may exist among classes and objects in Java.

7.1 Class Hierarchies

To begin to examine some of the relationships among classes and objects, we use an analogy. A dictionary might define a *Samoyed* as a dog of medium size with a thick white coat, erect ears, and a tail curling over the back. Even if you had never heard of a Samoyed before, these few words would give you a very good idea of what a Samoyed is. The key to giving such a clear picture with such brevity is the word *dog*. As soon as you know that a Samoyed is a kind of dog, you already have a great deal of information about it. All that the rest of the definition has to do is make clear what distinguishes a Samoyed from other dogs. We say that the Samoyed *inherits* characteristics from dogs.

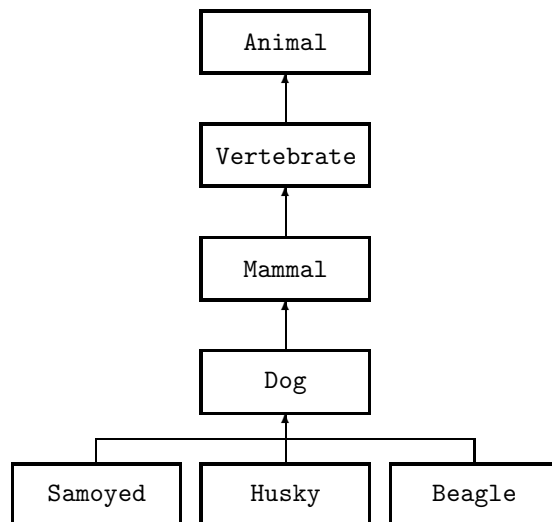
To take this idea closer to the meaning of inheritance in Java, let us consider `Dog` and `Samoyed` as classes. Then `Dog` is the *superclass* of `Samoyed`. Examining the relationship in the opposite direction, we can say that `Samoyed` is a *subclass* of `Dog` or `Samoyed extends Dog`. A class like `Dog` can have many subclasses. Subclasses of `Dog` would include `Husky`, `Beagle`, and so on. On the other hand, in Java, each class can have only one superclass.¹

Although a class can have no more than one superclass, it can inherit from many classes. Extending our example, we might say that the superclass of `Dog` is `Mammal`. Similarly, the superclass of `Mammal` might be `Vertebrate` and the superclass of `Vertebrate` might be `Animal` as shown in the next diagram.

Example 1

The diagram illustrates the class-hierarchy relationships of the classes that we have been discussing.

¹Some languages *do* permit classes to extend more than one class. This feature is known as *multiple inheritance*.



Each class in the diagram inherits from all the classes in the chain above it so that, for example, `Dog` inherits from `Animal`, `Vertebrate`, and `Mammal`. The top class, `Animal`, inherits from no other class. In Java, the class that has this property is the class `Object` in the package `java.lang`. All other classes inherit from `Object` — either directly, by extending `Object`, or indirectly, by extending some other class that inherits from `Object`. To indicate in Java that a class extends another class, we use the word `extends` in the class header. If `Samoyed` and `Dog` were Java classes, the definition of `Samoyed` would begin

```
class Samoyed extends Dog
```

If no superclass is specified by an `extends` clause, `Object` is the default superclass.

The classes from which a class inherits are sometimes referred to as its *ancestors*. The classes that inherit from a class are sometimes referred to as its *descendants*. Thus, in Example 1, the descendants of `Mammal` are `Dog`, `Samoyed`, `Husky`, and `Beagle` while the ancestors of `Mammal` are `Vertebrate` and `Animal`.

What does it mean to say that a class “inherits” from another class? Essentially, it means that the fields and methods of the superclass are, in a sense, part of the subclass. To illustrate the relationships of objects in a hierarchy, in this chapter we will be drawing objects in a style that is somewhat different from that used previously. To illustrate this new style,

we use an example that we will be referring to frequently.

Example 2

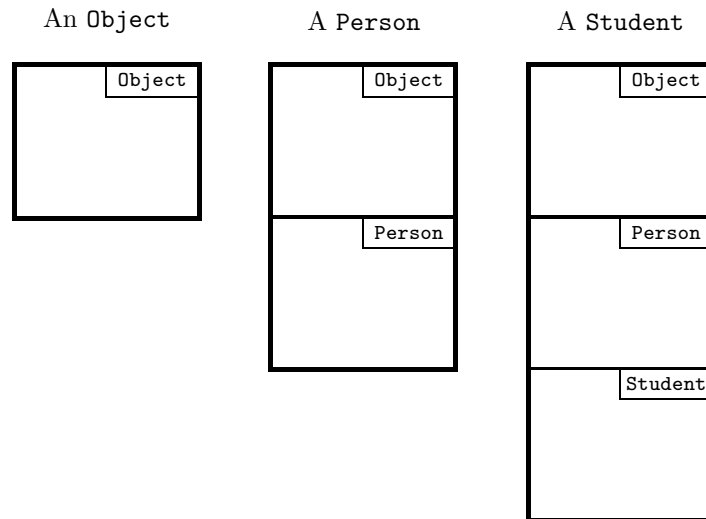
Suppose that we begin to define a class **Person** by writing

```
class Person
```

and that we begin to define a class **Student** by writing

```
class Student extends Person
```

Since we did not state that **Person** extends any class, then by default it extends the root class **Object**. Diagrams to illustrate objects of the classes **Object**, **Person**, and **Student** could take the following form.



Notice how the diagrams reflect the inheritance relations. Any **Person** object has both an **Object** part and a **Person** part. Any **Student** object has an **Object** part, a **Person** part, and a **Student** part.

Exercises 7.1

1. (a) In the class-hierarchy diagram shown in Example 1, what is the superclass of **Mammal**?
- (b) What class or classes does **Dog** extend?

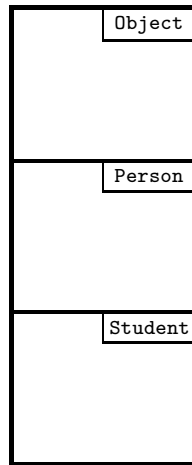
- (c) From what class or classes does **Husky** inherit?
 - (d) Extend the diagram to include **Invertebrate**, **Human**, and **Insect**.
 - (e) Suggest names of two other subclasses of **Vertebrate**.
2. If you were to write a definition of each of the following, what class would you extend? (Do *not* actually write the definitions.)
- (a) Carrot
 - (b) Bungalow
 - (c) Flute
 - (d) Neutrino
3. Does every Java class have a superclass? Explain.
4. Explain the difference between the statements “**ClassA** extends **ClassB**” and “**ClassA** inherits from **ClassB**”.
5. (a) Draw a diagram, like the one in Example 1, showing the hierarchical relationship among the following classes: **Rectangle**, **Polygon**, **Quadrilateral**, **Square**, **Object**, **Triangle**, and **Shape**.
- (b) Draw a diagram, like one of the diagrams in Example 2, to show the structure of a **Quadrilateral** object.

7.2 Inheritance and Variables

Among other things, inheritance forces us to reconsider the idea of a reference variable’s *type* and the types of values that can be assigned to it. In the previous chapter, if we declared a variable to be of type **Fraction**, then the only values that we assigned to it were **Fraction** objects. Now, however, we see that objects can have parts of different types. In such a case, Java allows us to have a reference variable of type *T* refer to any object that has a *T* part.

Example 1

If we have a **Student** object, as discussed in the previous section, it contains parts that are of three different types (**Object**, **Person**, and **Student**).



Here, all of the following statements are valid (assuming that the appropriate constructors exist).

```
Student s = new Student();
Person p = new Student(); // a Student has a Person part
Object o = new Student(); // a Student has an Object part
```

Since the type of a reference variable and the type of the object to which it refers need not be the same, we have to be careful with assignments. We can make assignments within the same hierarchy but only if they are appropriate. Using our **Student** class again, the following fragment is valid.

```
Student s = new Student();
Object o = s;
```

In the second statement, **s** refers to a **Student** object that has an **Object** part. Any assignment upward in a hierarchy is valid. On the other hand, the following fragment is not correct.

```
Object o = new Object();  
Student s = o;
```

The compiler objects to the second statement because the right hand side is of type `Object` and does not have a `Student` part. It is, therefore, wrong to attempt to have the variable `s`, of type `Student`, refer to the object `o`.

Things can get confusing if we create references to objects of different types. As an example, consider the fragment

```
Object o = new Student();  
Student s = o;
```

The first statement is valid (because a `Student` has an `Object` part) but the second statement presents the compiler with a problem because it is attempting to make an assignment of an `Object` (that might not have a `Student` part) to a variable of type `Student`. By looking at both lines of the fragment, we can see that, in fact, the variable `o` refers to an object that *does* have a `Student` part but the compiler does not look at previous lines of code and it produces an error message.

To get around such problems, we use a device that we have seen before — a *cast*. We first encountered the concept of a cast with primitive types. With such types, a cast is required for an assignment that involves a *narrowing* type conversion — a conversion from one type to a more restrictive type (such as `double` to `int`). A similar situation holds with assignments of objects of different types. First, we can never assign an object to a reference variable of another type unless the two types are in the same class hierarchy. Second, if an assignment is to be made to a different type within the same hierarchy, it must use a cast if the assignment is made to a more restrictive type of object (sometimes called *downcasting* because the target is lower in the hierarchy). The following fragment is both valid and acceptable to the compiler.

```
Object o = new Student();  
Student s = (Student)o;
```

The cast of `o` to a `Student` does not alter `o`. It simply reassures the compiler that it need not worry about the assignment by telling it that `o` refers to an object that contains a `Student` part.

Of course, when using a cast, it is important to be sure that the cast makes sense. To show what we mean, consider the next fragment.

```
Object o = new Object();  
Student s = (Student)o;
```

The cast in the second statement will make the compiler happy. However, since `o` is of type `Object`, it does not have a `Student` part and so it should *not* be assigned to `s`, a variable of type `Student`. Although the fragment will compile, when we attempt to execute the compiled code, it will throw a `ClassCastException` error.

As we saw in Chapter 6, fields that are declared with the `private` attribute can only be seen from within their own class. This still holds even for classes in the same hierarchy. If you want to work with a `private` field in another class in the same hierarchy, you must still use accessor and mutator methods. For situations in which you want subclasses to have direct access to fields of a class, Java offers another visibility modifier: `protected`. Any field declared with the `protected` attribute can be seen either in that class or any of its subclasses but not elsewhere.

Example 2

Suppose that a class definition starts as follows. (The dots indicate other features of the class that we have not shown here.)

```
class Sample
{
    public int a;
    float b;
    protected char c;
    private boolean d;
    ...
}
```

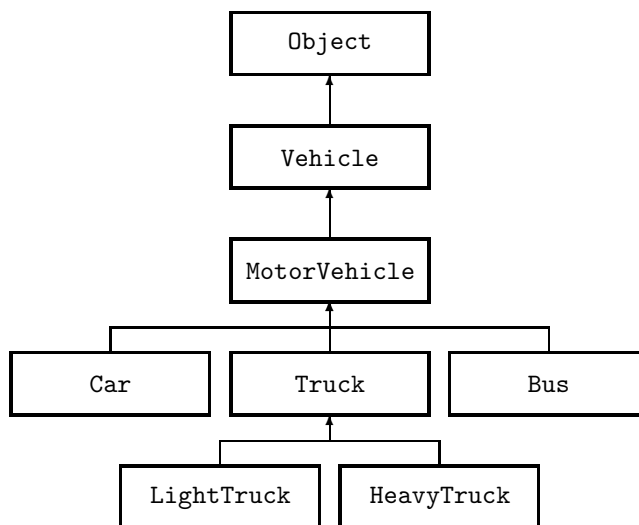
Here

- `a` is visible everywhere
- `b` is visible in the package in which `Sample` is defined
- `c` is visible within `Sample` and any of its subclasses
- `d` is visible only within `Sample`

As we have noted here, fields that are not declared to be **private** are, normally, visible from subclasses. If, however, we declare a field in a subclass that has the same identifier as a field in its superclass, then the field in the superclass is *shadowed* by the one in the subclass; it is no longer visible in the subclass. The two fields need not be of the same type; as long as the identifiers are the same, the field in the subclass shadows the other. This is very similar to the process of shadowing we saw in Chapter 6 with local variables shadowing fields in the same class.

Exercises 7.2

1. Consider the class hierarchy shown here and the statements that follow it. State, with reasons, which of the statements are valid and which are not. (Assume that appropriate constructors exist.)



- (a) `Vehicle v = new Car();`
- (b) `Truck t = new Truck();`
- (c) `Bus b = new MotorVehicle();`
- (d) `Object o = new Truck();`
- (e) `Car c = new Bus();`
- (f) `Bus b = new Vehicle();`

(g) `MotorVehicle m = new LightTruck();`

2. Referring to the classes shown in Example 1 of Section 7.1, which of the following conversions would require a cast?

- (a) Dog to Husky (b) Mammal to Animal
(c) Dog to Animal (d) Vertebrate to Beagle

3. Using the classes shown in Example 1 of Section 7.1, suppose that we construct the following variables.

```
Mammal pet = new Dog();  
Dog toby = new Samoyed();  
Dog sampson = new Dog();
```

Consider the following statements. For each one, state whether it is valid, incorrect but fixable with a cast, or not valid and not fixable with a cast. If a cast will make the statement valid, state the cast.

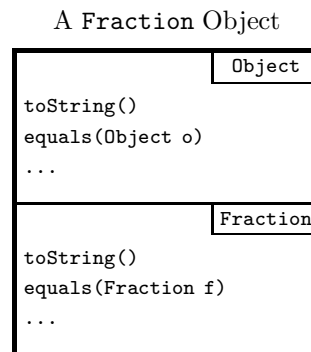
- (a) `Samoyed s = pet;` (b) `Object o = sampson;`
(c) `Samoyed s = toby;` (d) `Dog d = pet;`
(e) `Mammal m = toby;` (f) `Samoyed s = sampson;`

7.3 Inheritance and Methods

Although we did not use the term inheritance explicitly in Chapter 6, we certainly used some of its features. For example, we said that, for any class of objects that we define, Java provides a `toString` method for that class. Now we can see how this works. The `toString` method that Java supplies is defined in the class `Object`. Since all classes inherit from `Object`, objects of any type will always have an `Object` part and so they can all use the `toString` method defined there. In the same way, all classes can use the `equals` method defined in the `Object` class.

Recall that the default `toString` method (defined in the `Object` class) returned a string that contained the identifier of the class and a memory reference to the current object of the class. To make the `toString` method more useful, we could *override* the default definition in the `Object` class with one that had the same signature but whose functionality was more

appropriate. For example, we wrote a `toString` method for the `Fraction` class that returned the numerator and denominator of a fraction. We did the same sort of thing with the `equals` method, overriding the definition of `equals` in the `Object` class with our own, more useful definitions. The next diagram illustrates this for the `Fraction` class. The dots indicate methods that we have not shown.



Generally, when a call is made to an instance method, Java looks for a method with the appropriate signature starting from the lowest class in the hierarchy of the object. This is illustrated in the next example.

Example 1

If we were to write

```
Fraction f = new Fraction(2,3);
```

then the call `f.toString()` would invoke the `toString` method of the `Fraction` class, not the version in the `Object` class. If we had *not* written a `toString` method for the `Fraction` class, then Java would have started working its way up the class hierarchy looking for a method with the appropriate signature. In this case, the next class up the hierarchy is `Object` where the default version of `toString` resides.

As we noted in the previous section, the type of a reference variable need not be the same as the object to which it refers. When we use methods in such cases, we must be very careful to see which method is invoked. To illustrate this, let us look again at our `Object` – `Person` – `Student` hierarchy. Suppose now that we have written `equals` methods for the

classes `Person` and `Student`. Suppose further that we have constructed objects with the following statements.

```
Object o = new Student();
Person p = new Student();
```

Under these circumstances, an expression like

```
o.equals(p)
```

is valid. To decide which version of `equals` to use, Java uses the instance variable `o`. The type of the variable `o` is `Object` but the type of object that the variable `o` currently refers to is `Student`. Java uses the object's type (rather than the variable's type) to determine which method to call. As we said above, Java looks for a method with the appropriate signature starting from the lowest class in the hierarchy of the object. In this case, there is an `equals` method in the `Student` class so that is the one that Java uses.

In the preceding discussion, we supposed that a version of the `equals` method had been written for each of the classes `Object`, `Person`, and `Student`. Suppose now that we define a method with the header

```
public void foo ()
```

only in the `Student` class. If we now write

```
Person p = new Student();
p.foo();
```

then the compiler produces an error. The problem here is similar to the one that we encountered in the previous section; the compiler does not look at previous lines of code so it is not sure what type `p` will have at execution time. If it happens to have type `Person` at that time, that would be an error because there is no `foo` method in the `Person` class. The compiler does not want to take a chance on such an error occurring so it refuses to compile the code. We can solve the problem in the same way that we did previously — with a cast. If we write

```
Person p = new Student();
((Student)p).foo();
```

then the fragment will compile (and execute) correctly. The extra parentheses around `(Student)p` are necessary because a method call has a higher precedence than a cast and we want the cast applied to `p`, not to the result of evaluating `p.foo()`.

Exercises 7.3

1. Referring again to the `Object – Person – Student` hierarchy, suppose that we have a method defined in the `Person` class with the header

```
public String getName ()
```

and a method defined in the `Student` class with the header

```
public String getNumber ()
```

Now consider the following fragments from a `main` method. State which ones are valid, which ones are not valid but can be fixed by a cast, and which ones are not valid and cannot be fixed by a cast. Assume that all constructors are valid.

- (a) `Student s = new Student("Greg");`
`String name = s.getName();`
- (b) `Person p = new Person("Roberto");`
`String number = ((Student)p).getNumber();`
- (c) `Student s = new Student("Mona");`
`String name = ((Person)s).getName();`
- (d) `Person p = new Student("Emilio");`
`String number = p.getNumber();`
- (e) `Person p = new Student("Suzie");`
`String number = ((Student)p).getNumber();`
- (f) `Student s = new Student("Gaby");`
`String number = ((Student)s).getNumber();`
- (g) `Person p = new Student("Nicki");`
`String name = p.getName();`

2. What is the output of the following program?²

```
class A
{
    public A ()
        {System.out.println("A part");}
    public void m ()
        {System.out.println("A's m");}
}
```

²To save space, we have partially abandoned our usual indentation scheme.

```
class B extends A
{
    public B ()
    {System.out.println("B part");}
    public void m ()
    {System.out.println("B's m");}
}

class Sample
{
    public static void main (String[] args)
    {
        A a1 = new A();
        B b1 = new B();
        A a2 = (A)b1;
        a1.m();
        a2.m();
        b1.m();
    }
}
```

7.4 Using super and instanceof

Recall that, to refer to the current implicit object of a class, we used the reserved word **this**. In a similar way, we can refer to the superclass of a class by using the reserved word **super**. This is often used to invoke a constructor of the superclass. If, in a constructor of a class, we want to call the constructor of its superclass with some argument, we can do so. If such a call is present, it must be the very first statement in a constructor.

Example 1

Looking again at our **Person** and **Student** classes, suppose that we add the following field and constructor to the **Person** class. The dots indicate other features of the class that we have not shown here.

```
class Person
```

```
{
    private String name;
    public Person (String name)
    {
        this.name = name;
    }
    ...
}
```

Now, suppose that an object of the `Student` class has a student number as well as a name. We could modify our `Student` class to provide an appropriate constructor as follows.

```
class Student extends Person
{
    private String number;
    public Student (String name, String number)
    {
        super(name);
        this.number = number;
    }
    ...
}
```

The statement `super(name);` is a call to the constructor in `Person` with the `String` argument `name`. It will assign the appropriate value to the `name` field of the `Person` part of a `Student` object.

Even if we do not use `super` in a constructor, it is still lurking quietly in the background because every constructor that does *not* have a call to the constructor of the superclass automatically has the statement `super();` inserted as the first statement of the constructor. In this way, Java ensures that in constructing an object, any inherited fields are initialized.

The use of `super` is not restricted to constructors; it can be used in many ways. The next example shows how it can be used with `toString` methods.

Example 2

Suppose that we want to write `toString` methods for our `Person` and `Student` classes that we have been developing. The `toString` method of the `Person` class might return the value of the `name` field defined in that class. Thus, in `Person`, we might have

```
public String toString ()
{
    return "Name: " + name;
}
```

Since a `Student` is also a `Person`, we would probably want to have the `toString` method of the `Student` class return both the `name` (defined in the `Person` class) and the `student number` (defined in the `Student` class). Since the `name` field of the `Person` class is `private`, we cannot have direct access to it from the `Student` class but the `toString` method of the `Person` class is `public`. We can get access to this method from the `Student` class by using `super`.

```
public String toString ()
{
    return super.toString() + "    #: " + number;
}
```

Note in Example 2 that if, at some later stage, we decided to return birth dates along with names, we could simply change the body of the `toString` method in the `Person` class to

```
return "Name: " + name + "    Birth date: " + birthDate;
```

and the change would be seen in both the `Person` class and the `Student` class. No change would be required in the code of the `toString` method in the `Student` class.

We can also use `super` to gain access to shadowed fields or overridden methods in a superclass. Both of these ideas are illustrated in the next example.

Example 3

Suppose that the `Dog` class contained

```
class Dog
{
    protected String name = "Dog";
    public void feed ()
    {
        ...
    }
}
```

and the `Samoyed` class contained

```
class Samoyed extends Dog
{
    private String name = "Samoyed";
    public void feed ()
    {
        ...
    }
}
```

If we were writing code within the `Samoyed` class, and we wanted to use the `name` field of the `Dog` class, we could write `super.name` to do so. Similarly, the expression `super.feed()`, used in an instance method in the `Samoyed` class, would invoke the `feed` method of the `Dog` class for the current implicit `Samoyed` object.

Since reference variables can refer to objects in subclasses of their type and references can be altered during execution, we may not know the type of object currently referred to by a reference variable. In such cases, we can use the `instanceof` operator to help us.

Example 4

Suppose once again that the class `Student` is a subclass of the class `Person`. Suppose also that all `Person` objects have names (that are returned by the method `getName` in the `Person` class) but only `Student` objects have student numbers (that are returned by the method `getNumber` in the `Student` class). We can then write

```
Person p;
```

to create a variable of type `Person` that could refer to either a `Person` object or a `Student` object. After `p` has been assigned some value, we might use the following fragment to print information about the object referred to by `p`.

```
System.out.println(p.getName());
if (p instanceof Student)
    System.out.println(((Student)p).getNumber());
```

The last line illustrates a situation similar to one that we saw previously, in Section 7.3. Knowing from the second last line that `p` is currently referring to a `Student` object, we can safely print its student number. However, the compiler does not know this (since it does not look at previous code) so we must reassure it by casting `p` to a `Student`. The extra parentheses around `(Student)p` are required because a method call has a higher precedence than a cast.

Exercises 7.4

1. Explain the effect of placing the statement

```
super("man");
```

as the first statement of a constructor.
2. Suppose that a program contains the following classes.

```
class A
{
    public A () {System.out.print("A");}
}
class B extends A
{
    public B () {System.out.print("B");}
}
```

What, if anything, would be printed by each of the following statements if they appeared in the `main` method of the program?

(a) `A a = new A();`

```
(b) B b = new B();  
(c) B c = new A();
```

7.5 Polymorphism and the abstract Modifier

Returning once more to our `Object – Person – Student` hierarchy, suppose that we write

```
Person p1, p2;  
p1 = new Person(..);  
p2 = new Student(..);
```

If we then write

```
System.out.println(p1.toString());  
System.out.println(p2.toString());
```

in both cases, `p1` and `p2` are references of type `Person` but, at execution time, they refer to different types of objects (with different `toString` methods). This does not present a problem as Java decides at execution time which method is appropriate and uses that one automatically. A situation like this in which a different method is invoked depending on the type of object associated with the call at runtime, is an example of a feature called *polymorphism*, (Greek *polu*, many + *morphē*, form).

The last feature of class interaction that we will be looking at in this chapter is the use of abstract methods. As an example to illustrate this, suppose that we are writing a program to handle bank accounts. To do so we might create a class called `Account` that has fields and methods required by all accounts. We could then define classes `Savings` and `Chequing` that both extend `Account`. These classes would contain only the necessary additional fields and methods that are unique to either savings accounts or chequing accounts. If all account types give interest but different types of accounts have different rules for giving interest, a method `computeInterest` should be defined in each of the subclasses. As the system is developed, we may want to add other types of accounts. To ensure that all types will have a method to compute interest, we can *declare* a method `computeInterest` in the `Account` class but only *define* it in each subclass. To do so, we make the declaration in the superclass with the

modifier **abstract** and we provide no body for the method in that class. Instead, the various definitions are provided in the subclasses. If a class contains any methods that are declared to be abstract, then the class must also be declared to be abstract with the **abstract** modifier.

Example 1

To implement the ideas on interest computation outlined here, we could create the following structures.

```
abstract class Account
{
    abstract double computeInterest (int accountNumber);
    ...
}
class Savings extends Account
{
    double computeInterest (int accountNumber)
    {
        // implementation here
        ...
    }
    ...
}
class Chequing extends Account
{
    double computeInterest (int accountNumber)
    {
        // implementation here
        ...
    }
    ...
}
```

Although we can declare variables to have the type of an abstract class, we cannot create objects of such a class. Only the non-abstract subclasses of an abstract class can be instantiated. For a subclass of an abstract class to be non-abstract, it must provide actual definitions for *every* abstract

method in the abstract superclass. If it does not do so, then it must also be declared to be abstract (and it cannot be instantiated).

Abstract methods can help to simplify code by eliminating the need for casting. For the banking example, suppose that in a `main` method, we write the following fragment.

```
Account a = new Savings();
double interest = a.computeInterest();
```

Previously we saw fragments like this in which the compiler required a cast to assure it that the object is not of type `Account` (where there is no definition of the method). Here, however, the compiler will not object because there is no possibility that the object to which `a` refers could be of type `Account`. This is because the `Account` class is abstract and, therefore, it cannot be instantiated.

Exercises 7.5

1. If a method in a class is declared to be **abstract**, what else must be done?
2. What would be printed by the following program?

```
class A
{
    public void m1 ()
        {System.out.println("A1");}
}

class B extends A
{
    public void m1 ()
        {System.out.println("B1");}
    public void m2 ()
        {System.out.println("B2");}
}

class C extends B
{
    public void m2 ()
        {System.out.println("C2");}
}
```

```
class Driver
{
    public static void main (String[] args)
    {
        A a1 = new A();
        A a2 = new C();
        B b1 = new B();
        B b2 = new C();
        a1.m1();
        a2.m1();
        b1.m2();
        b2.m2();
    }
}
```

3. In the previous question, if we had added the statement
 `a2.m2();`
to the `main` method, the program would not compile.
- (a) Explain this.
 - (b) How could the statement be modified so that the program would compile and execute correctly?
 - (c) Without changing the statement, how could you use the **abstract** modifier so that the statement would no longer cause an error?
 - (d) If you were to implement the change in part (c), what new error would appear in the program?

7.6 Avoiding Errors and Debugging

Avoiding Errors

1. Try to avoid using the same code in different classes. Otherwise, if you want to alter the code, you will have to do so in each of the

classes in which the code appears. If this occurs in classes in the same hierarchy, it can frequently be avoided by using `super`.

As an example, suppose that a class `Toaster` is a subclass of `Appliance`. The `toString` method of the `Toaster` class might have the form

```
public String toString ()
{
    return super.toString()
        + " a toaster - capacity: " + maxSlices;
}
```

The `toString` method of `Appliance` would be responsible for returning a string that is appropriate for all appliances. Only the information specific to toasters would be added in the `Toaster` class.

2. Do not confuse method *overriding* (discussed in this chapter) with method *overloading* (introduced in Chapter 5). We override a method if we write another method, with the same signature as the original method, in a subclass of the original class. On the other hand we overload a method if we write more than one method with the same identifier but different signatures. The fact that overriding requires methods to have the same signature can cause errors. For example, suppose we have a class

```
class Sample
{
    public double foo (int n)
    {
        ...
    }
    ...
}
```

and the following subclass.

```
class Sub extends Sample
{
    public double foo (double n)
    {
```

```

        ...
    }
    ...
}

```

The method in the subclass does not have the same signature as the one in the superclass so it does not override it; it overloads it.

3. Many people argue that using **protected** visibility is an invitation to trouble. If a field is specified as being **private**, then it cannot be seen from outside its own class. If we have thoroughly tested the class, we can be reasonably sure that it will behave as we expect it to do. If, on the other hand, it is specified as being **protected**, then it can be altered from any subclass of the class in which it is declared. Such alteration might be done by someone other than the original writer and/or at a much later time than the original code was written. In either case, the value of the field could be easily corrupted. This can be avoided by always declaring fields as **private** and only permitting access to them by carefully controlled accessor and mutator methods.

Debugging

1. If a field does not have the correct value, look for inadvertent shadowing of that field. When a class inherits from another class, it may have additional fields but, normally, it will not have fields whose identifiers are the same as those in an ancestor class. For example, if we have

```

class Person
{
    protected String name;
    ...
}

```

and

```

class Student extends Person
{
    protected String name;
}

```

```

        protected int studentNumber;
        ...
    }

```

then, if we create a `Student` object called `Vincent`, the identifier `Vincent.name` will refer to the `name` field in the `Student` part of the object, not the `name` field in the `Person` part. While it is valid, it is probably not what we really intended.

Exercises 7.6

1. Study the following program and then answer the questions that follow it.

```

class A
{
    public void m (double n)
    {System.out.println(2*n);}
}
class B extends A
{
    public void m (int n)
    {System.out.println(3*n);}
}
class Driver
{
    public static void main (String[] args)
    {
        B b = new B();
        b.m(2);
        b.m(2.0);
    }
}

```

- (a) What would be printed by this program?
- (b) Is this an example of overriding or overloading?
- (c) What would the program print if `int` were replaced by `double`?
- (d) Explain your answer to part (c).

7.7 Review Exercises 7

1. Draw a diagram showing the hierarchical relationship among the following classes: `Bicycle`, `Car`, `Hatchback`, `Convertible`, `Truck`, and `Vehicle`.
2. Explain the meaning of the statement `super(value);` in a constructor method.
3. Explain the difference between a `private` method and a `protected` method.
4. What does it mean to have a method declared with the `abstract` modifier?
5. Explain the difference between method overriding and method overloading.
6. State whether or not each fragment of code is valid. If it is not valid, state why and note whether the error will be detected by Java during compilation or during execution. Assume that the constructors are valid.
 - (a) `Thing t = new Thing();`
`Object o = t;`
 - (b) `Object o = new Object();`
`Thing t = o;`
 - (c) `Object o = new Object();`
`Thing t = (Thing)o;`
 - (d) `Object o = new Thing();`
`Thing t = o;`
7. Suppose a program is to use the classes `Book` and `History` (extending `Book`).
 - (a) In which class should each of the following fields be declared?
 - `title`
 - `timePeriod`

- author
- (b) Write a constructor for the class `Book` with header
`public Book (String title, String author)`
- (c) Write a constructor for the class `History` with header
`public History (String title, String author,
String timePeriod)`
- (d) Write a `toString` method for the `Book` class.
- (e) Write a `toString` method for the `History` class.
8. What would be printed by the following program?

```
class A
{
    public void m ()
    {
        System.out.println("A");
    }
}

class B extends A
{
    public void m ()
    {
        super.m();
        System.out.println("B");
    }
}

class Driver
{
    public static void main (String[] args)
    {
        A a1 = new A();
        A a2 = new B();
        B b1 = new B();
        a1.m();
        a2.m();
        b1.m();
    }
}
```

9. What would be printed by the following program?

```
public class Menagerie
{
    public static void main(String[] args)
    {
        Pet p = new Cat("Gus");
        p.feed();
        p.stroke();
        System.out.println(p);
        Fish f = new Fish("Bubbles");
        f.feed();
        f.feed();
        System.out.println(p);
        System.out.println(f);
    }
}

abstract class Pet
{
    private String name;
    protected boolean alive = true;
    public Pet (String name)
    {
        this.name = name;
        System.out.print("New pet: " + name);
    }
    public abstract void feed ();
    public void stroke ()
    {
        System.out.println("mmm");
    }
    public String getStatus ()
    {
        if (alive)
            return "alive";
        else
            return "dead";
    }
    public String toString()
```

```
    {
        return name;
    }
}

class Cat extends Pet
{
    public Cat (String name)
    {
        super(name);
        System.out.println(" - a cat");
    }
    public void feed ()
    {
        System.out.println("meeow");
    }
    public void stroke ()
    {
        System.out.println("purr");
    }
    public String toString ()
    {
        return super.toString() + " is " + getStatus();
    }
}

class Fish extends Pet
{
    private int timesFed;
    public Fish (String name)
    {
        super(name);
        timesFed = 0;
        System.out.println(" - a fish");
    }
    public void feed ()
    {
        System.out.println("bloop");
        timesFed++;
        if (timesFed > 1)
```

```
        alive = false;
    }
    public String toString ()
    {
        return super.toString() + " is " + getStatus();
    }
}
```

10. Study the following classes (all of which compile correctly) and then answer the questions that follow them.

```
class Vehicle
{
    private int numWheels;
    public Vehicle (int nw)
    {
        numWheels = nw;
    }
    public int getNumWheels ()
    {
        return numWheels;
    }
}

class Truck extends Vehicle
{
    private int loadCapacity;
    public Truck (int nw, int lc)
    {
        super(nw);
        loadCapacity = lc;
    }
    public int getLoadCapacity ()
    {
        return loadCapacity;
    }
}

class Car extends Vehicle
{

```

```

    private int numSeats;
    public Car (int ns)
    {
        super(4);
        numSeats = ns;
    }
    public int getNumSeats ()
    {
        return numSeats;
    }
}

```

Consider each of the following fragments to be in a `main` method that uses these classes. All the constructors are valid. The other statements fall into one of the following categories.

1. The statement will compile and execute correctly.
2. The statement will not compile but it can be repaired by a cast.
3. The statement will not compile and cannot be repaired by a cast.
4. The statement will compile but fails to execute correctly.

For each fragment, identify the category to which the second statement belongs.

- (a) `Vehicle v1 = new Vehicle(4);`
`int nw = v1.getNumWheels();`
- (b) `Car c1 = new Car(6);`
`System.out.println(c1.getNumWheels());`
- (c) `Vehicle v2 = new Car(4);`
`int capacity = v2.getNumSeats();`
- (d) `Truck t1 = new Truck(18,14000);`
`int loadLimit = ((Vehicle)t1).getNumSeats();`
- (e) `Car c2 = new Car(5);`
`int maxLoad = (((Truck)c2).getLoadCapacity());`
- (f) `Vehicle v3 = new Vehicle(4);`
`System.out.println(((Car)v3).getNumSeats());`
- (g) `Vehicle v4 = new Truck(4,1200);`
`System.out.println(((Truck)v4).getNumWheels());`

```
(h) Truck t2 = new Truck(10,8000);  
    Car c3 = (Car)t2;
```

Projects

11. Write a set of classes that could be used to do simple calculations on geometric objects. The set should include the following classes.
 - (a) **Shape**, containing abstract methods **area** and **perimeter**
 - (b) **Rectangle**, a subclass of **Shape**, containing a constructor (with parameters for length and width) along with methods **area** and **perimeter**
 - (c) **Triangle**, a subclass of **Shape**, containing a constructor (with parameters for the three sides of a triangle) along with methods **area** (See page 84) and **perimeter**
 - (d) **Square**, a subclass of **Rectangle**, containing a constructor (with a parameter for the length of a side) and methods that invoke the methods of the **Rectangle** class to determine the perimeter and area of a square.